بسمه تعالی

📘 فایل آموزشی مصاحبه Backend

جلسه: Web Fundamentals & Backend Mindset

تفکیک‌شده برای دو سطح: Mid-Level و Senior

تهیه و تنظیم : علیرضا بهمنش راد - https://www.linkedin.com/in/behmaneshrad

---

بخش اول

🎓 آموزش سطح Mid-Level

این بخش تمام **اصطلاحات، مفاهیم و گزینه‌هایی** را که در سوالات مصاحبه Mid-Level استفاده شده، **تعریف و آموزش** می‌دهد.

1️⃣ API Contract (قرارداد API)

تعریف:

API یک **قرارداد رسمی** میان Backend و Client است که مشخص می‌کند:

- چه endpointهایی وجود دارند
- چه ورودی‌هایی پذیرفته می‌شود
- چه خروجی‌ای برگردانده می‌شود

نکته ذهنیتی:

API قرار نیست فقط امروز کار کند، باید **ماه‌ها و سال‌ها** بدون شکستن کلاینت‌ها دوام بیاورد.

---

2 Fragile API (API شکننده)

تعریف:

APIای که با **تغییرات کوچک** مثل:

- حذف فیلد
- تغییر اسم فیلد
- تغییر نوع داده

کلاینت‌ها را خراب می‌کند.

علت شکل‌گیری:

- طراحی عجولانه
- Mirror کردن دیتابیس
- نادیده گرفتن backward compatibility

---

3 Backward Compatibility (سازگاری با عقب)

تعریف:

یعنی نسخه جدید API، **کلاینت‌های قدیمی** را نشکند.

مثال:

✅ اضافه کردن یک فیلد optional

❌ حذف یا تغییر معنی فیلد قبلی

---

4 Resource-Oriented Design

تعریف:

طراحی API حول **منابع** (**Resource**) به‌جای فعل‌ها.

مثال صحیح:

- `/users/{id}`
- `/orders/{orderId}`

مثال اشتباه:

- `/createUser`
- `/cancelOrder`

---

5 HTTP Methods (GET, POST, PATCH, DELETE)

هدف هر متد:

- GET → خواندن
- POST → ایجاد
- PATCH → تغییر جزئی
- DELETE → حذف

نکته مهم:

استفاده‌ی نادرست از متدها باعث **API مبهم و غیرقابل فهم** می‌شود.

---

6 Stateless Backend

تعریف:

Backend نباید state کاربر (Session) را در memory خودش نگه دارد.

چرا مهم است؟

- Load Balancer درخواست را به هر سروری می‌فرستد
- اگر state محلی باشد → logout ناگهانی کاربران

راه‌حل‌ها:

- Redis
- Database
- JWT

---

7 Single Point of Failure (SPoF)

تعریف:

هر چیزی که فقط **یک نسخه** دارد و با failure آن کل سیستم down می‌شود.

مثال‌ها:

- یک سرور
- یک دیتابیس
- یک Redis بدون replica

---

8 Load Balancer

تعریف:

واسطی که درخواست‌ها را بین چند سرور توزیع می‌کند.

مزایا:

- افزایش availability
- امکان scale out
- حذف SPoF در لایه اپلیکیشن

Cache 9

تعریف:

ذخیره موقت داده‌های پرتکرار برای:

- افزایش سرعت
- کاهش فشار روی دیتابیس

انواع مهم:

- Browser Cache
- CDN Cache
- Application Cache (Redis)

TTL (Time To Live) 10

تعریف:

مدت زمانی که یک داده در cache معتبر است.

چرا مهم است؟

- جلوگیری از داده‌ی خیلی قدیمی
- حفظ تعادل بین سرعت و دقت

Cache Hit / Cache Miss 11

- ✅ **Hit** : داده در cache وجود دارد
- ❌ **Miss** : cache خالی است ← رجوع به DB

هدف:

افزایش Hit Rate (مثلاً بالای ۹۰٪)

1️⃣2️⃣ Scale Up vs Scale Out

Scale Up:

قوی‌تر کردن یک سرور (CPU/RAM بیشتر)

Scale Out:

اضافه کردن سرورهای بیشتر

✅ راه‌حل بلندمدت: **Scale Out**

✅ چک‌لیست سریع Junior-Level

اگر همه این‌ها رو می‌تونی با مثال ساده توضیح بدی، آماده‌ای:

- ☐ می‌دونم request و response دقیقاً چیه
- ☐ فرق GET و POST رو اشتباه نمی‌گم
- ☐ حداقل status codeهای معروف رو می‌فهمم
- ☐ می‌دونم client قابل اعتماد نیست
- ☐ مفهوم session و token رو می‌دونم
- ☐ می‌دونم REST چیه (در حد ساده)
- ☐ می‌دونم خطا رو چطور گزارش می‌کنیم
- ☐ می‌دونم HTTPS چرا مهمه

بخش دوم

🔥 آموزش سطح Senior

این بخش تمام مفاهیم **پیشرفته و ذهنیتی** سوالات Senior را پوشش می‌دهد.

1️⃣ Production-First Thinking

تعریف:

فکر کردن به رفتار سیستم در production **قبل از نوشتن کد**.

سوالات ذهنی Senior:

- اگر dependency کند شود؟
- اگر ۱۰ برابر load بیاید؟
- اگر سرویس downstream down شود؟

2️⃣ Stability vs Speed

تعریف:

سیستم پایدار مهم‌تر از سیستم سریع است.

نکته Release It:

API سریع بدون protection بزرگ → outage

---

3 Timeout

تعریف:

حداکثر زمانی که منتظر پاسخ می‌مانیم.

چرا حیاتی؟

- جلوگیری از گیر کردن thread
- محافظت از resourceها

قاعده:

هر timeout → call دارد (DB، HTTP، Queue)

---

4 Circuit Breaker

تعریف:

مکانیزمی شبیه فیوز برق:

- وقتی failure زیاد شود → مسیر قطع می‌شود
- بعد از مدتی دوباره تست می‌کند

هدف:

جلوگیری از **Cascading Failure**

---

5 Cascading Failure

تعریف:

خرابی یک سرویس → خرابی سرویس‌های دیگر → نابودی کل سیستم

علت اصلی:

- Retry بی‌حد
- Timeout نداشتن
- Pool محدود

---

6 Fail Fast

تعریف:

رد کردن سریع درخواست وقتی می‌دانیم موفق نمی‌شود.

چرا خوب است؟

- آزاد ماندن resource
- پاسخ قابل پیش‌بینی

---

7 Eventually Consistent

تعریف:

داده ممکن است موقتاً ناهماهنگ باشد ولی در نهایت همگرا می‌شود.

مثال‌ها:

- Replication
- Cache
- Distributed systems

---

8 Replication

تعریف:

کپی‌کردن دیتابیس برای توزیع read.

مزیت:

بهبود read scalability

ریسک:

Read قدیمی (lag)

---

9 Sharding

تعریف:

تقسیم داده‌ها بین چند دیتابیس مستقل.

چرا خطرناک است؟

- Join سخت
- Debug سخت
- Migration دردناک

قانون:

آخرین راه، نه اولین

---

10 Load Parameters

تعریف:

اعدادی که وضعیت واقعی سیستم را مشخص می‌کنند.

مثال‌ها:

- QPS
- Concurrent Users
- Data Size

Latency (P95 / P99) 1️⃣1️⃣

تعریف:

زمان پاسخ درخواست‌ها در بدترین حالت‌های قابل‌قبول.

چرا مهم‌تر از Average؟

- کاربران بدترین تجربه را حس می‌کنند نه میانگین را

Blast Radius 1️⃣2️⃣

تعریف:

میزان تأثیر خرابی یک بخش روی کل سیستم.

Senior Goal:

Blast Radius کوچک ✅

API Versioning 1️⃣3️⃣

تعریف:

مدیریت تغییرات ناسازگار API در طول زمان.

نکته مهم:

Versioning نشانه‌ی شکست قرارداد است، نه راه‌حل همیشگی.

---

14 Idempotency

تعریف:

اجرای چندباره‌ی یک درخواست ⟶ نتیجه یکسان

کاربرد حیاتی:

- پرداخت
- retry
- سیستم‌های توزیع‌شده

---

15 Saga Pattern

تعریف:

الگوی مدیریت transaction در microserviceها با rollback منطقی.

جایگزین:

Two-Phase Commit (که شکننده است)

✅ جمع‌بندی نهایی آموزشی

Mid-Level یعنی:

- API را قرارداد بداند
- Stateless طراحی کند
- Cache بشناسد
- SPoF را حذف کند

Senior یعنی:

- Production-First فکر کند
- Failure را پیش‌بینی کند
- Data consistency را بفهمد
- سیستم را به‌صورت end-to-end ببیند